KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

C PROJECT REPORT ON

STUDENTS ATTENDENCE MANAGEMENT SYSTEM

SUBMITTED TO

The Technical Department

KG College of Arts and Science

SUBMITTED BY

Student Name	Roll No.	Course&Batch
VARSHA V	25BIT159	B.SC IT 'B' (2025-2028)
VARSHINI V	25BIT160	B.SC IT 'B' (2025-2028)
VAUMSI KRISHNA A	25BIT161	B.SC IT 'B' (2025-2028)
VIJAYAKUMARI B	25BIT162	B.SC IT 'B' (2025-2028)
VISAL VELUSAMY	25BIT163	B.SC IT 'B' (2025-2028)

RAKESH N	25BIT164	B.SC IT 'B' (2025-2028)

Academic Year: **2025 – 2026**

Semester: **I**

DECLARATION

We hereby declare that the project entitled "**STUDENTS ATTENDENCE MANAGEMENT SYSTEM"** submitted in partial fulfilment of the requirements for the course [**C PROGRAMMING]** is a **record of our original work** carried out under the guidance of **[VASANTH M]**, Technical Trainer, Technical Department, KGiSL MicroCollege.

We further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

We have acknowledged all sources of information used in this project wherever applicable.

Student Name	Roll No.	Course&Batch	Signature
VARSHA V	25BIT159	B.SC IT 'B' (2025-2028)	
VARSHINI V	25BIT160	B.SC IT 'B' (2025-2028)	
VAUMSI KRISHNA A	25BIT161	B.SC IT 'B' (2025-2028)	
VIJAYAKUMARI B	25BIT162	B.SC IT 'B' (2025-2028)	
VISAL VELUSAMY	25BIT163	B.SC IT 'B' (2025-2028)	

RAKESH N	25BIT164	B.SC IT 'B' (2025-2028)	

Place: COIMBATORE

Date: _____

CERTIFICATE

This is to certify that the project entitled **"STUDENTS ATTENDENCE MANAGEMENT SYSTEM"** is a **bona fide work** carried out by the following students of **[BSC.IT "B" (2025-2028)]**, [**B.SC.INFORMATION AND TECHNOLOGY]**, **KG College of Arts&Science**, in partial fulfilment of the requirements for the course **[C PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **[MARCH,2026]**.

It is further certified that this project is the **original work** of the students and has not been previously submitted for the award of any degree, diploma, or certificate at this or any other institution.

Project Team:

S. No.	Name	Roll Number
1.	VARSHA V	25BIT159
2.	VARSHINI V	25BIT160
3.	VAUMSI KRISHNA A	25BIT161
4.	VIJAYAKUMARI B	25BIT162
5.	VISAL VELUSAMY	25BIT163
6	RAKESH N	25BIT164

Guide:
(Technical Trainer, KGM)

Head of the Department
(Technical Head, KGM)

Place:

Date:

ABSTRACT

The project titled **"Student Attendance Management System"** is developed using the C programming language to automate and manage student attendance efficiently. The primary objective of this system is to reduce manual effort in maintaining attendance records and to provide a simple, reliable, and organized method for tracking student attendance.

The system allows users to input student details such as ID and name, and then mark attendance as Present or Absent. It also provides options to view all student records and search for a specific student using their ID. Based on the attendance status, the system generates a remark indicating the student's attendance performance.

This project makes use of fundamental programming concepts in C, including structures for storing student data, arrays for handling multiple records, loops for iteration, and string handling functions for managing text data. Additionally, a union is used to store remarks efficiently during search operations.

The system follows a menu-driven approach, making it user-friendly and easy to operate. It is designed to be suitable for small-scale educational institutions where attendance tracking is required in a simple and efficient manner.

Overall, this project demonstrates the practical application of structured programming concepts and enhances understanding of data management, logical problem-solving, and real-world software development using the C programming language.

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	

3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

1 INTRODUCTION

.1 Objective

- To store student details such as ID and name using structures in C.
- To record and update attendance status (Present/Absent) for each student.
- To provide a menu-driven interface for managing attendance operations.
- To enable viewing of all student records along with their attendance.

- To search for a specific student by ID and display their attendance with a status remark using a union.

1.2 Overview

This system allows users to:

- Add and store student details such as ID and name.
- Mark attendance for each student as Present or Absent.
- View the attendance records of all students in a tabular format.
- Search for a specific student using their ID.
- Generate remarks based on the attendance status of each student

1.3 **Features**

- Menu-driven program for easy user interaction.
- Add, view, save, and load student records.
- Mark attendance (Present/Absent) for each student.
- Store attendance records date-wise using files.
- Generate attendance report with percentage calculation.

2.PROGRAM ANALYSIS

2.1 Problem Definition

A school or institution requires an efficient system to manage student attendance and maintain accurate records. Manual attendance tracking can be time-consuming and prone to errors, so an automated program is needed to simplify the process.

The system should be able to:

- Store student details such as ID (roll number) and name.
- Record daily attendance for each student as Present or Absent.
- Maintain and organize attendance records for multiple students.
- Allow users to view and search student attendance information easily.
- Calculate attendance status or related remarks based on recorded data.
- Provide a structured approach to managing attendance through a user-friendly system.

2.2 Project Overview

Aspect	Description
Language Used	C
Compiler	Dev-C++
Concepts Used	Structures, Arrays, Loops, Functions, Conditional Statements, File Handling
I/O Type	Console-based
Optional	Used to store student data (`students.csv`) and attendance records (`.att` files)

2.3 Scopes and Limitations

Scope:

- Suitable for schools or small institutions to manage student attendance efficiently.
- Can be extended to include advanced features such as database integration or a graphical user interface (GUI).
- Supports storing and managing attendance records for multiple students across multiple dates.
- Helps in automating attendance tracking and reducing manual errors.

Limitations:

- The program supports a maximum of 50 students only, limiting scalability.
- It is a console-based application and does not include a graphical user interface (GUI).
- There is no file handling, so all data is lost after the program ends.
- Attendance marking is completely manual and can be time-consuming for large inputs.
- The system does not validate duplicate student IDs or strictly check incorrect inputs.
- The remark using a union is temporary and not stored permanently for each student.

3.SYSTEM DESIGN

3.1 Modules

The Student Attendance Management System is divided into multiple functional modules. Each module is responsible for a specific task, and together they ensure smooth operation of the system.

Module Name	Description
Student Data Entry Module	This module is used to input and store student details such as student ID and name. It initializes the attendance status of each student.
Attendance Marking Module	This module allows the user to mark attendance for each student as Present or Absent. It updates the attendance record based on user input.
Attendance Display Module	This module displays all stored student records in a tabular format, including ID, name, and attendance status.
Student Search Module	This module enables the user to search for a specific student using their ID and displays their attendance details along with a status remark.
Remark Generation Module	This module uses a union to assign and display remarks such as "Good Attendance," "Needs Improvement," or "Not Marked" based on the student's attendance.
Menu Control Module	This module provides a menu-driven interface that allows users to navigate between different operations like marking attendance, viewing records, searching, and exiting the system.

Module Interaction Overview

- **Student Data Entry** → Enter and store student details.
- **Menu Control** → Display options to the user.
- **Attendance Marking** → Record Present/Absent status.
- **Attendance Display** → View all student records.
- **Student Search** → Search and display specific student details.
- **Remark Generation** → Generate remarks based on attendance.
- **Exit Module** → Terminate the program gracefully.

4.METHODOLOGY

The development of the **Student Attendance Management System** follows a structured methodology consisting of five main stages: Problem Analysis, System Design, Program Development, Testing & Validation, and Results & Documentation.

4.2. System Design

In this stage, the existing manual attendance system used in schools or institutions was studied and analyzed.
The main issues identified include manual errors in recording attendance, time consumption in maintaining registers, and difficulty in searching and maintaining records.

Based on these issues, the objectives of the system were defined, such as:

- Storing student details (ID and name).
- Marking attendance as Present or Absent.
- Viewing and managing attendance records efficiently.
- Searching for student details using ID.
- Generating remarks based on attendance status.

The problem was divided into smaller tasks to ensure modular and efficient program development.

4.3. Program Development

This is the implementation phase where the system design is converted into a working program using the C programming language. The code is developed using modular programming concepts, where the system is divided into smaller functional parts such as student data entry, attendance marking, attendance display, searching, and remark generation.

Control structures like loops, conditional statements (if-else), and switch-case are used to control program flow and handle user input. Arrays are used to store multiple student records, while structures are used to organize related data such as student ID, name, and attendance status. A union is used to generate and display remarks based on attendance.

4.4. Testing and Validation

After development, each module of the system is tested individually to ensure correct functionality. Various test cases are created to check different scenarios such as valid inputs, invalid choices, searching for existing and non-existing student IDs, and marking attendance for all students.

The system is tested by verifying the correctness of attendance marking, display output, and search functionality. The results are checked manually to ensure that the program behaves as expected and produces accurate outputs. After thorough testing, the system is validated to perform reliably without errors in attendance management operations.

4.5. Result and Documentation

Finally, the program produces the desired output by successfully managing student attendance records in a structured and organized manner. It allows users to mark attendance, view all student details, and search for specific students efficiently. The system also displays the attendance status along with appropriate remarks based on each student’s record.

Proper documentation has been prepared for the project, including the algorithm, flowchart, module descriptions, sample input/output screens, and conclusion. This ensures that the project is easy to understand, maintain, and further enhance in the future.

4.6. Algorithm

An algorithm is a step-by-step logical procedure used to solve a specific problem or perform a computation. In programming, it defines the exact sequence of actions the computer must follow to achieve the desired output from a given set of inputs.

For the **Student Attendance Management System**, the algorithm defines how the program accepts student details, records attendance, processes the data, and displays the results.

Purpose of the Algorithm

The main purpose of using an algorithm in this project is to:

- Define the logical flow of attendance management operations.
- Ensure accurate recording and retrieval of student attendance.
- Provide a structured approach for program development before coding.
- Make the system easier to understand, debug, and enhance in the future.

Algorithm in the Project

The algorithm designed for the Student Attendance Management System performs the following steps:

1. Start the program and initialize variables.
2. Input the number of students.
3. For each student, enter and store student ID and name.
4. Display a menu with available options.
5. Accept user choice from the menu.
6. If the user selects attendance marking:
    - For each student, input attendance as Present or Absent.
    - Store the attendance status.
7. If the user selects view attendance:
    - Display all student details along with attendance status.
8. If the user selects search:

- Accept student ID and search for the record.
- Display student details and generate a remark based on attendance.

9. Repeat the menu until the user chooses to exit.
10. Display an exit message and terminate the program.

Importance of Algorithm in the Project

- Helps convert real-world attendance management into structured program steps.
- Ensures systematic flow of control and logical correctness.
- Acts as a blueprint for coding and debugging.
- Improves clarity, modularity, and efficiency of the system.

4.7. Flowchart

The flowchart represents the logical flow of the Student Attendance Management System. It visually outlines the sequence of operations — from entering student details to marking attendance, searching records, and displaying results — ensuring clarity in how data is processed at each stage of the program.

Explanation of Flow

1. **Start** – Program begins execution.
2. **Enter Student Details** – Input student ID and name for all students.
3. **Display Menu** – Shows available options such as mark attendance, view records, search, and exit.
4. **User Selection** – The user selects an option from the menu.
5. **Mark Attendance** –

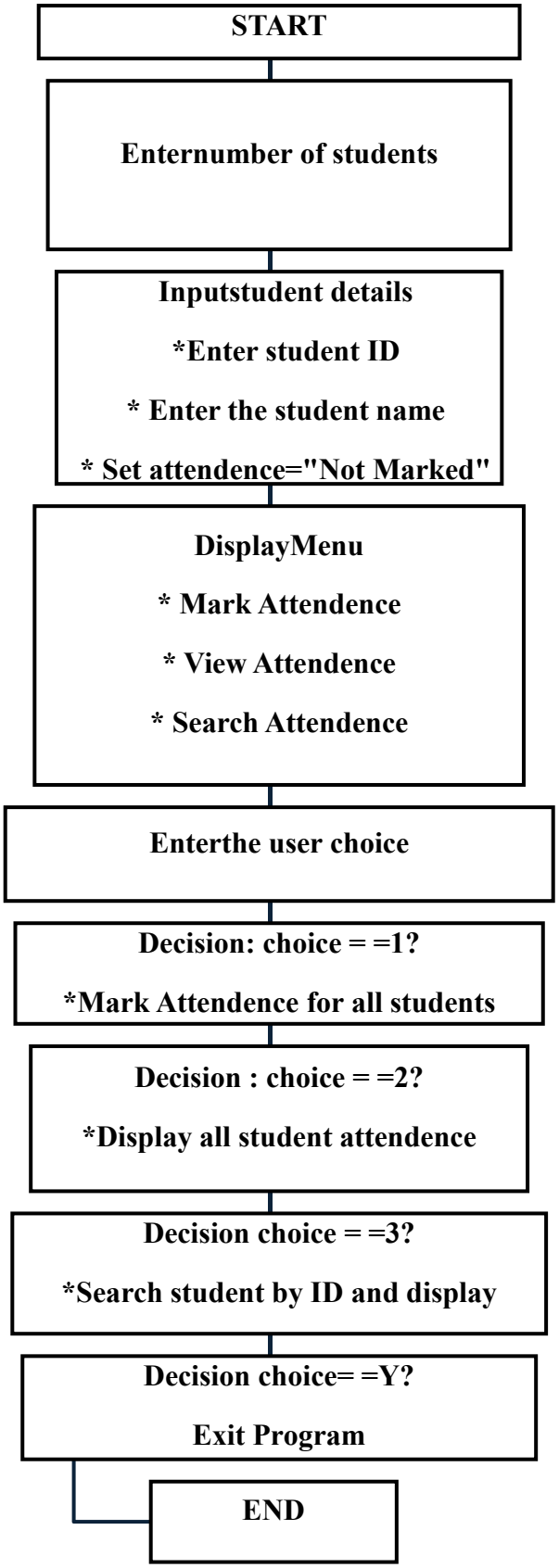
    - Input attendance (Present/Absent) for each student.
    - Store and update attendance status.

6. **View Attendance** – Display all student records with ID, name, and attendance.
7. **Search Student** –

    - Enter student ID.
    - Search and display the student's details.
    - Generate and display attendance remark.

8. **Repeat Menu** – Return to the menu for further operations if needed.

9. **End** – Exit the program after the user chooses to terminate.



1. SOURCE CODE

```
#include<stdio.h>
#include<string.h>

struct Student {
    int id;
    char name[50];
    char attendance[15];
};

union Status {
    char remark[25];
    char flag;
};

int main() {
    struct Student s[50];
    union Status st;
    int count, i, option;
    char choice;

    printf("Enter number of students (max 50): ");
    scanf("%d",&count);
    getchar();

    for (i = 0; i < count; i++) {
        printf("\nEnter details of student %d:\n", i + 1);
```

```
    printf("Enter Student ID: ");
    scanf("%d",&s[i].id);
    getchar();

    printf("Enter Student Name: ");
    fgets(s[i].name, sizeof(s[i].name), stdin);
    s[i].name[strcspn(s[i].name, "\n")] = '\0';

    strcpy(s[i].attendance, "Not Marked");
}

do {
    printf("\n===== STUDENT ATTENDANCE MANAGEMENT =====\n");
    printf("1. Mark Attendance\n");
    printf("2. View Attendance\n");
    printf("3. Search Student\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d",&option);
    getchar();

    switch (option) {

        case 1: {
            int j = 0;
            while (j < count) {
                char mark;
                printf("Mark attendance for %s (P/A): ", s[j].name);
                scanf(" %c",&mark);
```

```
        if (mark == 'P' || mark == 'p')
            strcpy(s[j].attendance, "Present");
        else if (mark == 'A' || mark == 'a')
            strcpy(s[j].attendance, "Absent");
        else
            strcpy(s[j].attendance, "Invalid");

        j++;
    }
    printf("\nAttendance marked successfully!\n");
    break;
}

case 2:
    printf("\nID\tName\t\tAttendance\n");
    printf("-------------------------------------\n");

    for (i = 0; i < count; i++) {
        printf("%d\t%-15s%s\n", s[i].id, s[i].name, s[i].attendance);
    }
    break;

case 3: {
    int id, found = 0;
    printf("Enter Student ID to search: ");
    scanf("%d",&id);

    for (i = 0; i < count; i++) {
```

```
        if (s[i].id == id) {
            found = 1;

            printf("\nStudent Found:\n");
            printf("ID: %d\nName: %s\nAttendance: %s\n",
                   s[i].id, s[i].name, s[i].attendance);

            if (strcmp(s[i].attendance, "Present") == 0)
                strcpy(st.remark, "Good Attendance");
            else if (strcmp(s[i].attendance, "Absent") == 0)
                strcpy(st.remark, "Needs Improvement");
            else
                strcpy(st.remark, "Not Marked");

            printf("Status Remark: %s\n", st.remark);
            break;
        }
    }

    if (!found)
        printf("Student with ID %d not found!\n", id);

    break;
}

case 4:
    printf("Exiting system...\n");
    break;
```

```
        default:
            printf("Invalid choice! Please try again.\n");
    }

    printf("\nDo you want to continue? (y/n): ");
    scanf(" %c",&choice);

  } while (choice == 'y' || choice == 'Y');

  printf("Thank you for using the Student Attendance Management System!\n");
  return 0;
}
```

SOURCE CODE EXPLANATION:

1) Headers&Macros

- ```
  #include<stdio.h>
  #include<stdlib.h>
  #include<string.h>
  ```

- `stdio.h` is used for input and output functions like `printf()` and `scanf()`.
- `stdlib.h` provides general utility functions (included for completeness).
- `string.h` is used for string operations such as `strcpy()` and `strcmp()`.

2) Data Models (Structures)

```
struct Student {
    int id;
    char name[50];
    char attendance[15];
};
```

- The `Student` structure is used to store student-related information.
- It contains:
  - `Id` → Unique identification number of the student.
  - `Name` → Name of the student.
  - `Attendance` → Stores attendance status such as “Present”, “Absent”, or “Not Marked”.

```
union Status {
    char remark[25];
    char flag;
};
```
```

- The `Status` union is used to store remarks related to attendance.
- It helps in displaying messages like:
- "Good Attendance"
- "Needs Improvement"
    - "Not Marked"
    - Since it is a union, it shares memory among its members, making it efficient.

3) Main Function — Variables&Menu Setup

```
struct Student s[50];
union Status st;
int count, i, option;
char choice;
```

- `s[50]` is an array of structures used to store details of up to 50 students.
- `st` is a union used to store and display remarks related to attendance.
- `count` stores the number of students entered by the user.
- `i` is used as a loop counter for iteration.
- `option` stores the user's menu choice.
- `choice` is used to decide whether the user wants to continue or exit the program.

4) Print Title&Menu

```
Printf("\n==============================\n");
printf(" \n STUDENT ATTENDANCE MANAGEMENT
Printf(\n==============================\n");
printf("\nMenu:\n");...
for (i = 0; i < count; i++)
    printf("%d\t%-15s%s\n", s[i].id, s[i].name, s[i].attendance);
```

- Displays a formatted title for the system to make the interface user-friendly.
- Shows a tabular list of student records including ID, name, and attendance status.
- `%-15s` ensures the name is left-aligned, improving readability in column format.
- The loop iterates through all stored students and prints their details neatly.

5) Ask "Number of Students"

- `printf("Enter number of students (max 50): ");`
  `scanf("%d",&count);`
- Accepts the total number of students to be entered into the system.
- `count` determines how many student records will be stored and processed.
- The limit (max 50) ensures controlled memory usage and prevents overflow.

Example:
If `count = 3`, the system will take input for 3 students and manage their attendance accordingly.

6) Loop to Capture Each Student's Attendance

```
for (i = 0; i < count; i++) {
    char mark;

    printf("Mark attendance for %s (P/A): ", s[i].name);
    scanf(" %c",&mark);

    if (mark == 'P' || mark == 'p')
        strcpy(s[i].attendance, "Present");
    else if (mark == 'A' || mark == 'a')
        strcpy(s[i].attendance, "Absent");
    else
        strcpy(s[i].attendance, "Invalid");
}
```

- Iterates through each student one by one.
- Takes attendance input as P (Present) or A (Absent).
- Validates input and stores the corresponding status.
- Updates each student's attendance in the array.

7) Attendance Status / Remark Generation

```
if (strcmp(s[i].attendance, "Present") == 0)
    strcpy(st.remark, "Good Attendance");
else if (strcmp(s[i].attendance, "Absent") == 0)
    strcpy(st.remark, "Needs Improvement");
else
    strcpy(st.remark, "Not Marked");
```

- Checks the attendance status of a student.
- Assigns a remark using the union `Status`.
- Provides meaningful feedback based on attendance.

8) Display Student Attendance Records

```
printf("\nID\tName\t\tAttendance\n");
printf("-------------------------------------\n");

for (i = 0; i < count; i++) {
    printf("%d\t%-15s%s\n", s[i].id, s[i].name, s[i].attendance);
}
```

- Displays all stored student records in a tabular format.
- Shows each student's ID, name, and attendance status.
- Uses formatted output to align data neatly for better readability.
- Helps users quickly review attendance of all students.

9) Search and Display Specific Student

```
printf("Enter Student ID to search: ");
scanf("%d",&id);

for (i = 0; i < count; i++) {
    if (s[i].id == id) {
        printf("\nStudent Found:\n");
        printf("ID: %d\nName: %s\nAttendance: %s\n",
               s[i].id, s[i].name, s[i].attendance);

        if (strcmp(s[i].attendance, "Present") == 0)
            strcpy(st.remark, "Good Attendance");
        else if (strcmp(s[i].attendance, "Absent") == 0)
            strcpy(st.remark, "Needs Improvement");
        else
            strcpy(st.remark, "Not Marked");

        printf("Status Remark: %s\n", st.remark);
        break;
    }
}
```

- Accepts a student ID from the user.
- Searches the student list to find a matching record.
- Displays the student's details if found.
- Generates a remark based on attendance using a union.
- Helps in identifying individual student performance.

10) Program Exit

```
return 0;
```

- Ends the execution of the program.
- Indicates successful termination of the attendance system.

Control Flow Summary

1. Display system menu and options.
2. Input student attendance (Present/Absent).
3. Store and update attendance records.
4. View all student records in tabular form.
5. Search for a student using ID and display details.
6. Generate attendance remarks.
7. Repeat until the user chooses to exit.
8. Terminate the program successfully.

6.INPUT AND OUTPUT SCREENS

Output Screen:

```
C:\Users\KGCAS\OneDrive\Dc

Enter Student ID: 102
Enter Student Name: meena

===== STUDENT ATTENDANCE MANAGEMENT =====
1. Mark Attendance
2. View Attendance
3. Search Student
4. Exit
Enter your choice: 1
Mark attendance for varsha (P/A): P
Mark attendance for meena (P/A): A

Attendance marked successfully!

Do you want to continue? (y/n): y

===== STUDENT ATTENDANCE MANAGEMENT =====
1. Mark Attendance
2. View Attendance
3. Search Student
4. Exit
Enter your choice: 2

ID      Name            Attendance
-------------------------------------
101     varsha          Present
102     meena           Absent

Do you want to continue? (y/n): n
Thank you for using the Student Attendance Management System!
```

7.DISCUSSION&CONCLUSION

7.1 Discussion

This program implements a Student Attendance Management System using structured programming in C. It demonstrates the use of structures, unions, arrays, loops, and switch-case statements.

The system allows users to:
- Store student details
- Mark attendance as Present or Absent
- View all student records
- Search for a specific student by ID

The use of arrays of structures enables efficient storage of multiple student records, while the union is used to store status remarks during search operations.

The program follows a menu-driven approach, making it user-friendly and interactive. It is suitable for small-scale attendance management systems and demonstrates real-world application of C programming concepts.

7.2 Conclusion

The project successfully fulfills its objective of creating a simple and functional Student Attendance Management System.

It demonstrates practical implementation of core C programming concepts such as:
- Structures for data organization
- Arrays for handling multiple records
- Unions for memory-efficient data usage
- Looping and decision-making constructs

The system is easy to use, efficient, and can be further enhanced with additional features such as file handling, login authentication, and database integration. It provides a strong foundation for building more advanced management systems.

8.REFERENCES

Kernighan, B. W.,&Ritchie, D. M. (1988).
The C Programming Language, Prentice Hall.

2. TutorialsPoint – C Programming Tutorials
https://www.tutorialspoint.com/cprogramming/

3. GeeksforGeeks – C Programming Examples
https://www.geeksforgeeks.org/c-programming-language/

4. Standard C Library Documentation